GRUPO 3 · REQUERIMIENTOS FUNCIONALES

Préstamos

Módulo: Préstamos, cuotas, saldo pendiente, vencimientos y estado

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 3
Versión	1.0
Fecha	10 de octubre de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST, `/api/v2`) y aiquaa-sandbox-web (front, `/v2`)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training_v2
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el ciclo de vida de un préstamo: la solicitud, la modificación de sus condiciones mientras está en evaluación, la aprobación —que fija el saldo pendiente y genera el plan de cuotas—, la consulta del cronograma, el pago de cuotas hasta cancelar el préstamo, y la baja.

Fuera de alcance: El desembolso del dinero y el débito de las cuotas (ninguna operación del módulo mueve el saldo de una cuenta); el rechazo de una solicitud; la evaluación crediticia; el interés por mora y el vencimiento automático de cuotas; los pagos parciales o adelantados con recálculo; y la amortización con sistema francés o alemán (el sandbox usa interés simple con cuota fija).

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con la API key de su grupo (o con un token JWT de grupo) y automatiza escenarios BDD sobre el módulo de su grupo.
Cliente del banco (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es titular de cuentas, tarjetas, préstamos, beneficiarios, planes de ahorro y depósitos.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo sobre el schema `qa_training_v2`, bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz tipo home banking que consume la API a través del proxy `/api/proxy/v2/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v2/prestamos	RF-G3-01	Lista préstamos vigentes, filtrados por titular y/o estado.
POST /api/v2/prestamos	RF-G3-02	Registra una solicitud de préstamo.
GET /api/v2/prestamos/{id}	RF-G3-03	Devuelve el detalle de un préstamo.
PUT /api/v2/prestamos/{id}	RF-G3-04	Reemplaza las condiciones de un préstamo solicitado.
DELETE /api/v2/prestamos/{id}	RF-G3-05	Da de baja un préstamo sin saldo pendiente.
POST /api/v2/prestamos/{id}/aprobar	RF-G3-06	Aprueba el préstamo y genera el plan de cuotas.
GET /api/v2/prestamos/{id}/cuotas	RF-G3-07	Devuelve el cronograma de cuotas, opcionalmente por estado.
POST /api/v2/prestamos/{id}/cuotas/{numero}/pagar	RF-G3-08	Paga una cuota y descuenta el saldo pendiente.
POST /api/v2/g3/auth/token	RF-G3-TK	Emite el token JWT del grupo.

2. Precondiciones y acceso

- **Base URL de la API:** `https://aiquaa-sandbox-api.vercel.app`. La documentación interactiva del curso está en `/docs/v2` y la especificación OpenAPI en `GET /api/v2/docs`. La aplicación web está en `https://aiquaa-sandbox-web.vercel.app`.
- Toda request debe autenticarse por una de dos vías: el header `x-api-key` con una clave activa de `public.api_keys`, o el header `Authorization: Bearer <token>` con un JWT de grupo vigente. Si se envían ambos, manda `x-api-key`. Sin credenciales, o con credenciales inválidas, inactivas o vencidas, la respuesta es `401 UNAUTHORIZED` con un mensaje genérico que no indica cuál fue el motivo.
- Las rutas bajo `/api/v2` solo aceptan credenciales del **curso 2**: una API key válida pero del curso 1 recibe `403 FORBIDDEN` con el mensaje "Esta API key pertenece al curso 1 y esta ruta es del curso 2.". El gate es simétrico: una key del curso 2 tampoco sirve en `/api/v1`.
- Cada credencial admite un máximo de **30 requests por minuto** (ventana deslizante, contada por API key o por usuario del token, no por IP). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training_v2` (10 tablas) y se cargan con `scripts/seed-data-v2.sql`: 8 clientes (más un cliente por grupo, usado por el login de token), 12 cuentas, 20 movimientos, 10 tarjetas, 6 préstamos con sus cuotas, 10 beneficiarios, 12 transferencias, 6 planes de ahorro y 6 depósitos. La data inicial es determinística, pero **el sandbox es compartido por los cinco grupos**: los registros sembrados pueden haber sido modificados por otra suite. Un escenario robusto crea sus propios datos en lugar de depender del estado de una fila sembrada.
- **Clientes de soporte:** `GET|POST /api/v2/usuarios` y `GET|PUT|DELETE /api/v2/usuarios/{id}` no pertenecen a ningún grupo y están disponibles para todos; sirven para crear el titular de los productos de cada escenario. `GET /usuarios` acepta el filtro `email`; `POST` exige `nombre`, `email` y `documentoNumero` (email y documento son únicos: repetirlos da `409 CONFLICT`).
- **Sandbox SQL:** `POST /api/v2/sql/select` y `POST /api/v2/sql/update` reciben `{ "sql", "params" }` y permiten consultar o preparar datos directamente sobre las 10 tablas del curso. Admiten un único statement por request, con placeholders `$1..$n`; el `UPDATE` exige cláusula `WHERE`. Son la vía para verificar en base lo que hizo un endpoint, o para forzar un estado que la API no produce.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` hace baja lógica (`activo = false`, o `activa = false` en `cuentas`) y devuelve `204 No Content` sin cuerpo. Una fila dada de baja deja de verse en `GET`, `PUT` y `DELETE`, pero sigue existiendo en la base.
- En la aplicación web el acceso tiene tres capas, en orden: elegir el curso (`/curso`, opción 2), cargar la API key del curso (`/login`, que valida que la key sea del curso elegido) y elegir el cliente con el que se opera (`/auth/login`, por email de un cliente activo). Las pantallas `/v2/usuarios/*` son la excepción a la tercera capa: se pueden usar solo con la API key, porque son el punto de partida para crear un cliente.
- Para solicitar un préstamo se necesita un cliente existente (`usuarioId`); la cuenta de acreditación (`cuentaId`) es opcional y solo informativa.
- Datos sembrados: préstamos 1 y 5 en `solicitado` (sin cuotas); préstamos 2, 3 y 4 en `aprobado` con cuotas; préstamo 6 en `pagado` con sus tres cuotas pagadas. El préstamo 4 tiene dos cuotas en estado `vencida`.

- **Los préstamos sembrados 2 y 3 no cierran contablemente** (ver brechas): su saldo pendiente no coincide con la suma de sus cuotas impagas. Para un escenario que pague un préstamo hasta cancelarlo, hay que crear y aprobar un préstamo propio.
- Las fechas de vencimiento de las cuotas se calculan a partir del día en que se aprueba: un escenario no debe esperar fechas fijas, sino fechas relativas a la de ejecución.

3. Modelo de datos del módulo

prestamos

Préstamos solicitados y otorgados. `estado` es el estado de negocio y `activo` la marca de baja lógica.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`cuenta_id`	`bigint`, opcional, referencia a `cuentas(id)`
`monto_solicitado`	`numeric(14,2)`, obligatorio, mayor que cero
`tasa_interes`	`numeric(5,2)`, obligatoria, mayor o igual a cero; porcentaje total, no anual
`plazo_meses`	`integer`, obligatorio, entre 1 y 120
`saldo_pendiente`	`numeric(14,2)`, obligatorio, por defecto 0; mayor o igual a cero
`estado`	`text`, uno de `solicitado` / `aprobado` / `rechazado` / `pagado`; por defecto `solicitado`
`activo`	`boolean`, obligatorio, por defecto `true`; `false` = dado de baja
`created_at`	`timestamptz`, por defecto `now()`

cuotas_prestamo

Plan de cuotas de cada préstamo aprobado. No tiene marca de baja lógica: las cuotas no se eliminan.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`prestamo_id`	`bigint`, obligatorio, referencia a `prestamos(id)`
`numero_cuota`	`integer`, obligatorio, mayor que cero; único dentro del préstamo
`monto`	`numeric(14,2)`, obligatorio, mayor que cero
`fecha_vencimiento`	`date`, obligatoria
`estado`	`text`, uno de `pendiente` / `pagada` / `vencida`; por defecto `pendiente`
`fecha_pago`	`timestamptz`, opcional; se completa al pagar
`created_at`	`timestamptz`, por defecto `now()`

Fórmulas del módulo, con interés simple aplicado una sola vez sobre el capital: **total a devolver** = monto × (1 + tasa / 100), redondeado a dos decimales; **cuota** = total / plazo, redondeada a dos decimales; la **última cuota** absorbe la diferencia de redondeo para que la suma de las cuotas sea exactamente el total. La tasa no es anual: un préstamo al 18 % devuelve el 118 % del capital sea cual sea el plazo. Ejemplo: 2.000.000 al 16 % en 3 meses da un total de 2.320.000, con cuotas de 773.333,33, 773.333,33 y 773.333,34.

4. Requerimientos funcionales

RF-G3-01 — Listar préstamos

`GET /api/v2/prestamos`

Devuelve los préstamos vigentes del sandbox, con la posibilidad de acotar por titular y por estado.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `solicitado`, `aprobado`, `rechazado` o `pagado`.

Reglas de negocio

- Devuelve únicamente préstamos no dados de baja (`activo = true`).
- Los dos filtros se combinan entre sí.
- El listado no incluye las cuotas: se consultan por préstamo con RF-G3-07.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de préstamos; cada elemento incluye `id`, `usuario_id`, `cuenta_id`, `monto_solicitado`, `tasa_interes`, `plazo_meses`, `saldo_pendiente`, `estado`, `activo` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` no es un entero positivo, o `estado` no es uno de los cuatro valores permitidos.

Fuente: `app/api/v2/prestamos/route.ts`

RF-G3-02 — Solicitar un préstamo

`POST /api/v2/prestamos`

Registra una solicitud de préstamo. La solicitud queda en evaluación, sin saldo pendiente y sin cuotas.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `cuentaId` (opcional): número entero positivo de la cuenta de acreditación.
- `montoSolicitado` (obligatorio): número mayor que cero, como número JSON.
- `tasaInteres` (obligatorio): número entre 0 y 999,99, como número JSON. Una tasa de 0 es válida.
- `plazoMeses` (obligatorio): número entero entre 1 y 120, como número JSON.

Reglas de negocio

- El préstamo nace con `estado = 'solicitado'`, `saldo_pendiente = 0` y sin cuotas: el plan lo genera la aprobación (RF-G3-06).
- El titular debe existir, y la cuenta también si se indica; un id inexistente es rechazado por la clave foránea.
- **No se valida que la cuenta pertenezca al titular, ni su estado ni su moneda**; tampoco que el titular esté activo.
- No hay tope de monto de negocio, ni límite de préstamos simultáneos por cliente.

Respuesta esperada

- `201 Created` con `data` conteniendo el préstamo creado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, el monto no es positivo, la tasa es negativa o supera 999,99, el plazo no es un entero entre 1 y 120, algún valor numérico llegó como texto, o el titular o la cuenta no existen.

Fuente: `app/api/v2/prestamos/route.ts`

RF-G3-03 — Consultar un préstamo

`GET /api/v2/prestamos/{id}`

Devuelve el detalle de un préstamo puntual: condiciones, saldo pendiente y estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve el préstamo en cualquier estado de negocio, sin sus cuotas.
- Si no existe o está dado de baja, responde 404 con el mensaje "Préstamo no encontrado.".

Respuesta esperada

- `200 OK` con `data` conteniendo el préstamo completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un préstamo vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/prestamos/[id]/route.ts`

RF-G3-04 — Modificar las condiciones de un préstamo

`PUT /api/v2/prestamos/{id}`

Reemplaza monto, tasa y plazo de un préstamo que todavía está en evaluación.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `montoSolicitado` (obligatorio): número mayor que cero.
- `tasaInteres` (obligatorio): número entre 0 y 999,99.
- `plazoMeses` (obligatorio): número entero entre 1 y 120.

Reglas de negocio

- Solo se puede modificar un préstamo en estado `solicitado`. En cualquier otro estado responde 409 con el mensaje "Un préstamo en estado 'aprobado' ya no puede modificarse." (con el estado que corresponda): una vez aprobado ya existe un plan de cuotas emitido contra esos números.
- Es un reemplazo: los tres campos son obligatorios aunque solo cambie uno.
- El titular, la cuenta y el estado no forman parte del cuerpo; si se envían, se ignoran.

Respuesta esperada

- `200 OK` con `data` conteniendo el préstamo ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un préstamo vigente con ese identificador.
`CONFLICT`	409	El préstamo ya no está en estado `solicitado`.
`VALIDATION_ERROR`	400	Falta un campo o alguno está fuera de rango.

Fuente: `app/api/v2/prestamos/[id]/route.ts`

RF-G3-05 — Dar de baja un préstamo

`DELETE /api/v2/prestamos/{id}`

Da de baja lógica un préstamo que no tiene saldo pendiente.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Un préstamo con saldo pendiente mayor que cero no puede darse de baja: responde 409 con el mensaje "No se puede eliminar un préstamo con saldo pendiente.".
- En la práctica se pueden dar de baja los préstamos `solicitado` (desistir de la solicitud) y los `pagado`; uno `aprobado` solo cuando su saldo llegó a cero.
- Marca `activo = false`; el préstamo y sus cuotas permanecen en la base, pero las cuotas dejan de poder consultarse por la API.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un préstamo vigente con ese identificador.
`CONFLICT`	409	El préstamo tiene saldo pendiente.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/prestamos/[id]/route.ts`

RF-G3-06 — Aprobar un préstamo

`POST /api/v2/prestamos/{id}/aprobar`

Aprueba una solicitud: pasa el préstamo a `aprobado`, fija el saldo pendiente en el total a devolver y emite el plan de cuotas, todo en una sola operación.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo. No lleva cuerpo.

Reglas de negocio

- Solo se aprueba un préstamo en estado `solicitado`. En otro estado responde 409 con el mensaje "El préstamo ya está en estado 'aprobado'." (con el estado que corresponda).
- El saldo pendiente queda en el total a devolver: monto × (1 + tasa / 100), redondeado a dos decimales.
- Se generan tantas cuotas como meses de plazo, numeradas desde 1, todas en estado `pendiente`.
- Cada cuota vale total / plazo redondeado a dos decimales; la última absorbe la diferencia de redondeo, de modo que la suma de las cuotas es exactamente el saldo pendiente.
- Cada cuota vence tantos meses después de la fecha de aprobación como indica su número: la primera un mes después, la segunda dos, y así sucesivamente.
- La operación es atómica: dos aprobaciones simultáneas no pueden generar el plan dos veces.
- **La aprobación no acredita el dinero**: el saldo de la cuenta asociada no cambia y no se registra ningún movimiento.

Respuesta esperada

- `200 OK` con `data` conteniendo el préstamo actualizado y, en `data.cuotas`, el arreglo con el plan completo recién generado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un préstamo vigente con ese identificador.
`CONFLICT`	409	El préstamo no está en estado `solicitado`.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/prestamos/[id]/aprobar/route.ts`

RF-G3-07 — Consultar el cronograma de cuotas

`GET /api/v2/prestamos/{id}/cuotas`

Devuelve el plan de cuotas de un préstamo, con la posibilidad de ver solo las de un estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `estado` (opcional, por query): `pendiente`, `pagada` o `vencida`.

Reglas de negocio

- Primero verifica que el préstamo exista y no esté dado de baja; si no, responde 404 en lugar de una lista vacía.
- Devuelve las cuotas ordenadas por número de cuota ascendente, sin tope de cantidad.
- Un préstamo `solicitado` todavía no tiene cuotas: devuelve una lista vacía.
- El filtro usa el estado guardado: una cuota `pendiente` con fecha de vencimiento pasada no aparece como `vencida`.

Respuesta esperada

- `200 OK` con `data` como arreglo de cuotas; cada elemento incluye `id`, `prestamo_id`, `numero_cuota`, `monto`, `fecha_vencimiento`, `estado`, `fecha_pago` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un préstamo vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo, o `estado` no es uno de los tres valores permitidos.

Fuente: `app/api/v2/prestamos/[id]/cuotas/route.ts`

RF-G3-08 — Pagar una cuota

`POST /api/v2/prestamos/{id}/cuotas/{numero}/pagar`

Registra el pago de una cuota: la marca como pagada, descuenta su importe del saldo pendiente y, si el saldo llega a cero, deja el préstamo cancelado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo del préstamo.
- `numero` (obligatorio, en la ruta): número entero positivo. Es el **número de cuota** dentro del préstamo, no el `id` de la cuota. No lleva cuerpo.

Reglas de negocio

- El préstamo debe estar en estado `aprobado`. En otro estado responde 409 con el mensaje "El préstamo está en estado 'solicitado': no admite pagos." (con el estado que corresponda).
- Si el préstamo no tiene una cuota con ese número, responde 404 con el mensaje "Cuota no encontrada.".
- Una cuota ya pagada no se puede volver a pagar: responde 409 con el mensaje "La cuota ya fue pagada.".
- Tanto una cuota `pendiente` como una `vencida` se pueden pagar, por su importe original: no hay recargo por mora.
- La cuota queda con `estado = 'pagada'` y `fecha_pago` con el momento del pago.
- El saldo pendiente del préstamo baja en el importe de la cuota (nunca por debajo de cero); cuando llega a cero, el préstamo pasa a `pagado`.
- **Las cuotas se pueden pagar en cualquier orden**: la API no exige pagar primero la más antigua.
- **El pago no debita ninguna cuenta**: no se valida saldo ni se registra movimiento. Siempre se paga la cuota completa.
- La operación es atómica: dos pagos simultáneos de la misma cuota no pueden descontar el saldo dos veces.

Respuesta esperada

- `200 OK` con `data.cuota` (la cuota ya pagada) y `data.prestamo` (el préstamo con su saldo pendiente y estado actualizados).

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe el préstamo, o no tiene una cuota con ese número.
`CONFLICT`	409	El préstamo no está aprobado, o la cuota ya fue pagada.
`VALIDATION_ERROR`	400	El identificador o el número de cuota no son enteros positivos.

Fuente: `app/api/v2/prestamos/[id]/cuotas/[numero]/pagar/route.ts`

RF-G3-TK — Obtener el token del grupo

`POST /api/v2/g3/auth/token`

Intercambia el usuario y la contraseña del grupo por un token JWT que puede usarse en lugar de la API key en cualquier ruta del curso. La ruta en sí también exige estar autenticado (API key del curso 2 o un token vigente).

Entradas y validaciones

- `username` (obligatorio): texto de 1 a 64 caracteres. El usuario del grupo es `c2_g03_prestamos`.
- `password` (obligatorio): texto de 1 a 200 caracteres. Se entrega por el canal del curso; no se publica en este documento.

Reglas de negocio

- La contraseña se compara en la base de datos contra un hash bcrypt; nunca se devuelve ni se registra en claro.
- Usuario inexistente, contraseña incorrecta y credencial inactiva devuelven el mismo `400 VALIDATION_ERROR` con el mensaje "Usuario o password inválidos.": la respuesta no revela cuál de los tres falló. Es `400` y no `401` a propósito, para no confundirlo con el fallo de la API key.
- Si las credenciales son válidas pero pertenecen a otro grupo, la respuesta es `403 FORBIDDEN` con el mensaje "Estas credenciales pertenecen al grupo 4 y esta ruta es del grupo 3.".
- El token es un JWT firmado con HS256, emitido por `aiquaa-sandbox`, con vigencia de **3600 segundos**. Sus claims (`sub`, `usuarioId`, `grupo`, `curso`) son legibles: el token está firmado, no cifrado.
- El token identifica al grupo pero no restringe por grupo: con él se puede operar sobre los recursos de cualquier módulo del curso 2.
- La emisión no deja rastro en tablas de dominio (el curso 2 no tiene tabla de sesiones); solo queda la fila de auditoría.

Respuesta esperada

- `200 OK` con `data` conteniendo `token`, `tokenType` (`Bearer`), `expiresIn` (`3600`), `grupo` (`3`), `grupoNombre`, `curso` (`2`) y `usuario` (`id`, `nombre`, `email`, `username`).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `username` o `password`, están vacíos, o la combinación no es válida.
`FORBIDDEN`	403	Las credenciales son de otro grupo, o la API key usada para llamar es del curso 1.
`UNAUTHORIZED`	401	La request no trae API key ni token.

Fuente: `app/api/v2/g3/auth/token/route.ts`, `lib/group-token-route.ts` y `lib/jwt.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de cohorte (curso) → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Tipos en la respuesta:** los identificadores (`id`, `usuario_id`, `cuenta_id`, ...) viajan como **número**; los importes y tasas (`saldo`, `monto`, `limite_credito`, `tasa_anual`, ...) viajan como **texto con dos decimales** (por ejemplo `"5000000.00"`), por ser valores numéricos exactos de la base; las fechas sin hora viajan como `YYYY-MM-DD` y las marcas de tiempo (`created_at`, `fecha_pago`) en formato ISO 8601. Una aserción sobre un importe debe comparar contra texto o convertir a número antes.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo). Los campos no declarados en el esquema se ignoran sin error.
- **Coerción de tipos:** los identificadores (`id`, `usuarioId`, `cuentaId`, ...) se convierten de texto a número automáticamente, vengan por query, por ruta o en el cuerpo. Los **importes, tasas y plazos del cuerpo JSON deben enviarse como número**: enviarlos como texto (`"monto": "1000"`) devuelve `400 VALIDATION_ERROR`.
- **Cuerpo JSON:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body."; un cuerpo vacío se interpreta como `{}`. Un fallo del esquema devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid request." y el detalle del campo en `error.details`.
- **Reglas de negocio → 409:** cuando el cuerpo es válido pero el estado actual del recurso no permite la operación (saldo insuficiente, recurso en un estado que no admite la acción), la respuesta es `409 CONFLICT` con un mensaje en español que describe la regla. `400` queda para entradas inválidas.
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`, por ejemplo un `usuarioId` inexistente) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos (una fecha imposible, un número fuera de rango) cae en `400 EXECUTION_ERROR`. En los tres casos el `message` es el texto crudo de Postgres.
- **Importes con más de dos decimales:** la API no valida la cantidad de decimales; la base redondea a dos al guardar.
- **Operaciones que mueven dinero:** se ejecutan en una transacción con bloqueo de las filas involucradas; si alguna validación falla, no queda ningún cambio parcial (ni saldo modificado ni movimiento registrado).
- **Listados:** devuelven como máximo 100 filas ordenadas por identificador ascendente, sin paginación ni indicación de que hubo recorte. Los filtros opcionales se combinan entre sí (`AND`).
- **Headers de caudal:** toda respuesta que pasó el control de caudal incluye `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` (este último en milisegundos Unix).
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera el framework y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** las requests quedan registradas en `public.sql_audit_log` con la credencial usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Sin control de titularidad:** la API no tiene roles ni noción de "mis" productos. Cualquier credencial válida del curso puede leer y modificar los recursos de cualquier cliente, y ningún endpoint verifica que la cuenta indicada pertenezca al `usuarioId` indicado.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso, y las acciones que cambian un estado sin crear un registro (bloquear, activar, aprobar, pagar una cuota, anular, cancelar).
`201 Created`	POST que crea un recurso o registra un movimiento de dinero nuevo.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Faltan credenciales, la API key no existe o está inactiva, o el token es inválido o venció.
`FORBIDDEN`	403	La credencial es válida pero es de otro curso, o las credenciales de un grupo se usaron en la ruta de token de otro grupo.
`RATE_LIMITED`	429	Se superaron las 30 requests por minuto de esa credencial.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe o ya está dado de baja.
`CONFLICT`	409	Regla de negocio violada sobre un recurso que existe, o violación de una restricción `UNIQUE`.
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/v2/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú muestra los módulos del grupo del alumno (más "Clientes", siempre visible) cuando su email figura en el roster del curso; si no figura, muestra todos. Cada listado arranca filtrado por el cliente con el que se inició sesión y permite cambiar a otro titular o a "todos"; la búsqueda, el orden y la paginación de los listados se resuelven en el navegador. **El front es deliberadamente más estricto que la API en varios puntos**: esas reglas están marcadas como "regla de UI" y solo se pueden probar por pantalla — la misma operación enviada directo a la API es aceptada. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
`/v2/prestamos` — Listado de préstamos	Tabla con número, titular, monto, tasa, saldo pendiente, avance de pago y estado; filtros por titular y por estado, con la deuda total y la cantidad de solicitudes en evaluación. Ejecuta RF-G3-01.
`/v2/prestamos/new` — Simular y solicitar	Asistente de dos pasos: en "Simulación" se cargan titular, cuenta, monto, plazo e interés, y se ve el cronograma proyectado; en "Confirmación" se revisa el resumen y se aceptan las condiciones. Ejecuta RF-G3-02 y navega al detalle.
`/v2/prestamos/{id}` — Detalle de préstamo	Muestra condiciones, saldo pendiente y avance. Mientras está `solicitado` permite aprobar (RF-G3-06) y modificar condiciones (RF-G3-04) con la simulación a la vista; una vez aprobado muestra el cronograma (RF-G3-07) y permite pagar cuotas (RF-G3-08). También permite dar de baja (RF-G3-05).

- **Regla de UI:** el monto debe estar entre 500.000 y 500.000.000 de guaraníes. La API acepta cualquier monto mayor que cero.
- **Regla de UI:** el plazo se elige de una lista fija (6, 12, 18, 24, 36, 48 o 60 meses). La API acepta cualquier entero de 1 a 120.
- **Regla de UI:** la cuenta de acreditación es obligatoria y debe ser una cuenta activa en guaraníes del titular. La API la acepta opcional y sin validarla.
- **Regla de UI:** solo se puede pagar la próxima cuota impaga, en orden. La API acepta pagar cualquier cuota.
- **Regla de UI:** la solicitud exige aceptar las condiciones del préstamo; ese dato no se envía a la API.
- La simulación de la pantalla usa la misma fórmula que la aprobación, por lo que el cronograma proyectado coincide con el que genera la API si se aprueba el mismo día.
- El botón de baja se llama "Desistir de la solicitud" mientras el préstamo está `solicitado` y queda deshabilitado mientras haya saldo pendiente.
- El avance de pago se calcula en el navegador como 1 − saldo pendiente / total; no es un dato de la API.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G3-01 — Listar préstamos

- Al listar sin filtros, la respuesta es 200 y `data` es un arreglo ordenado por `id` ascendente.
- Al filtrar por `estado=solicitado`, todos los elementos devueltos tienen `estado` igual a `solicitado` y `saldo_pendiente` igual a `"0.00"`.
- Al filtrar por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- Al filtrar por `estado=rechazado`, la respuesta es 200 con un arreglo vacío (ningún dato sembrado ni endpoint produce ese estado).
- Al enviar `estado=vigente`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G3-02 — Solicitar un préstamo

- Al solicitar un préstamo válido, la respuesta es 201, `data.estado` es `solicitado` y `data.saldo_pendiente` es `"0.00"`.

- Inmediatamente después de solicitar, RF-G3-07 sobre ese préstamo devuelve un arreglo vacío.
- Al solicitar con `plazoMeses` igual a 0, 121 o 6.5, la respuesta es 400 `VALIDATION_ERROR`.
- Al solicitar con `plazoMeses` igual a 1 y a 120 (los límites), la respuesta es 201.
- Al solicitar con `montoSolicitado` igual a 0 o negativo, la respuesta es 400 `VALIDATION_ERROR`.
- Al solicitar con `tasaInteres` igual a 0, la respuesta es 201; con −1 o 1000, es 400 `VALIDATION_ERROR`.
- Al solicitar para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ningún préstamo.

RF-G3-03 — Consultar un préstamo

- Dado un préstamo existente, al consultarlo la respuesta es 200 y `data.id` coincide con el solicitado.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Préstamo no encontrado.".
- Al consultar con un identificador no numérico, la respuesta es 400 `VALIDATION_ERROR`.

RF-G3-04 — Modificar las condiciones de un préstamo

- Dado un préstamo `solicitado`, al cambiar su plazo la respuesta es 200 y el cambio se refleja al volver a consultarlo.
- Dado un préstamo `aprobado`, al intentar modificarlo la respuesta es 409 `CONFLICT` y sus condiciones no cambian.
- Dado un préstamo `pagado`, al intentar modificarlo la respuesta es 409 `CONFLICT`.
- Al enviar solo `montoSolicitado`, la respuesta es 400 `VALIDATION_ERROR`.
- Si se modifica un préstamo y luego se aprueba, el plan de cuotas se calcula con las condiciones modificadas.

RF-G3-05 — Dar de baja un préstamo

- Dado un préstamo `solicitado`, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, RF-G3-03 y RF-G3-07 sobre ese préstamo responden 404, y deja de aparecer en RF-G3-01.
- Dado un préstamo `aprobado` con saldo pendiente, al intentar darlo de baja la respuesta es 409 `CONFLICT` y el préstamo sigue siendo consultable.
- Dado un préstamo cancelado por completo (`pagado`), al darlo de baja la respuesta es 204.

RF-G3-06 — Aprobar un préstamo

- Dado un préstamo `solicitado` de 6.000.000 al 20 % en 6 meses, al aprobarlo la respuesta es 200, `data.estado` es `aprobado`, `data.saldo_pendiente` es `"7200000.00"` y `data.cuotas` tiene 6 elementos de `"1200000.00"`.
- Dado un préstamo de 2.000.000 al 16 % en 3 meses, las cuotas son `"773333.33"`, `"773333.33"` y `"773333.34"`, y su suma es exactamente 2.320.000.
- Todas las cuotas generadas tienen `estado` igual a `pendiente` y `fecha_pago` nula, con `numero_cuota` consecutivo desde 1.
- La `fecha_vencimiento` de la primera cuota es un mes posterior a la fecha de aprobación, y cada cuota siguiente vence un mes después de la anterior.
- Dado un préstamo con tasa 0, el saldo pendiente es igual al monto solicitado.

- Al aprobar por segunda vez el mismo préstamo, la respuesta es 409 `CONFLICT` y no se generan cuotas adicionales.
- Tras aprobar un préstamo con cuenta asociada, el saldo de esa cuenta es el mismo que antes.
- Al aprobar un préstamo inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G3-07 — Consultar el cronograma de cuotas

- Dado un préstamo recién aprobado a 6 meses, la respuesta es 200 y `data` tiene 6 cuotas ordenadas por `numero_cuota`.
- Dado un préstamo `solicitado`, la respuesta es 200 y `data` es un arreglo vacío.
- Después de pagar una cuota, al filtrar por `estado=pagada` se devuelve exactamente esa cuota, y al filtrar por `estado=pendiente` se devuelven las restantes.
- Al enviar `estado=anulada`, la respuesta es 400 `VALIDATION_ERROR`.
- Al pedir las cuotas de un préstamo inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Préstamo no encontrado.".

RF-G3-08 — Pagar una cuota

- Dado un préstamo recién aprobado con saldo 7.200.000 y cuotas de 1.200.000, al pagar la cuota 1 la respuesta es 200, `data.cuota.estado` es `pagada`, `data.cuota.fecha_pago` no es nula y `data.prestamo.saldo_pendiente` es `"6000000.00"`.
- Al pagar por segunda vez la misma cuota, la respuesta es 409 `CONFLICT` con el mensaje "La cuota ya fue pagada." y el saldo pendiente no cambia.
- Al pagar todas las cuotas de un préstamo propio, tras el último pago `data.prestamo.saldo_pendiente` es `"0.00"` y `data.prestamo.estado` es `pagado`.
- Dado un préstamo `pagado`, al intentar pagar cualquier cuota la respuesta es 409 `CONFLICT`.
- Dado un préstamo `solicitado`, al intentar pagar la cuota 1 la respuesta es 409 `CONFLICT`.
- Al pagar la cuota 99 de un préstamo de 6 cuotas, la respuesta es 404 `NOT_FOUND` con el mensaje "Cuota no encontrada.".
- Al pagar la cuota 3 antes que la 1 y la 2, la respuesta es 200: la API no impone orden.
- Dada una cuota sembrada en estado `vencida`, al pagarla la respuesta es 200 y el saldo baja exactamente en el `monto` de la cuota.

RF-G3-TK — Obtener el token del grupo

- Con el usuario y la contraseña del grupo 3, la respuesta es 200, `data.tokenType` es `Bearer`, `data.expiresIn` es 3600 y `data.grupo` es 3.
- Una request posterior a cualquier endpoint del módulo enviando solo `Authorization: Bearer <token>` (sin `x-api-key`) responde con éxito.
- Con una contraseña incorrecta, la respuesta es 400 `VALIDATION_ERROR` con el mensaje "Usuario o password inválidos."; con un usuario inexistente, la respuesta es idéntica.
- Con las credenciales válidas del grupo 4 sobre esta ruta, la respuesta es 403 `FORBIDDEN`.
- Con un token alterado o vencido en el header `Authorization`, cualquier endpoint responde 401 `UNAUTHORIZED`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre transparencia e información de operaciones de crédito	Antes de contratar, el cliente debe conocer el monto total a pagar, la tasa, el plazo y el importe de cada cuota.	Parcial: la web muestra la simulación completa antes de solicitar, pero la tasa del sandbox es un porcentaje total y no una tasa anual efectiva, ni se informa costo financiero total.
Normativa del BCP sobre clasificación de activos, riesgos crediticios y previsiones	El otorgamiento requiere evaluar la capacidad de pago del deudor, y la cartera debe clasificarse según su atraso.	No implementado: cualquier solicitud se puede aprobar sin evaluación, y las cuotas no pasan a `vencida` por sí solas.
Ley 2339/03 y normativa del BCP sobre tasas máximas de interés (usura)	La tasa de interés pactada no puede superar el tope vigente publicado por el BCP.	No implementado: la API acepta cualquier tasa hasta 999,99 %.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El deudor debe poder consultar en todo momento su saldo y el estado de sus cuotas, y recibir constancia de cada pago.	Cubierto: la API devuelve saldo pendiente, cronograma y la cuota pagada con su fecha, y la web los muestra en el detalle.
Buenas prácticas de administración de créditos	El desembolso y el cobro de cuotas deben reflejarse como movimientos en la cuenta del cliente.	No implementado: aprobar no acredita la cuenta y pagar no la debita; el módulo no genera movimientos.

Brechas conocidas del sandbox

- No hay desembolso: aprobar un préstamo no acredita el dinero en la cuenta asociada ni registra ningún movimiento.
- Pagar una cuota no debita ninguna cuenta ni valida fondos: el pago es puramente registral.
- No existe endpoint para rechazar una solicitud: el estado `rechazado` está en el modelo pero ni la API ni los datos sembrados lo producen.
- El estado `vencida` de una cuota solo aparece en los datos sembrados: la API nunca lo asigna, y no hay interés por mora.
- Las cuotas se pueden pagar en cualquier orden; la pantalla web sí impone el orden.
- **Préstamo sembrado 2:** su saldo pendiente (11.500.000) es menor que la suma de sus 11 cuotas impagas (12.705.000). Al pagar cuotas, el saldo llega a cero y el préstamo pasa a `pagado` antes de la última cuota, y las cuotas restantes quedan `pendiente` sin poder pagarse.
- **Préstamo sembrado 3:** es a 24 meses pero solo tiene 4 cuotas cargadas, y su saldo pendiente (18.000.000) supera con creces a las cuotas impagas: nunca puede llegar a `pagado` por la API.

- La tasa es un porcentaje total aplicado una sola vez, no una tasa anual: el costo del préstamo no depende del plazo.
- `POST /prestamos` no valida que la cuenta pertenezca al titular, ni su estado ni su moneda, y no limita el monto ni la cantidad de préstamos por cliente.
- No hay pagos parciales, adelantos de capital ni cancelación anticipada con quita de intereses.
- Al dar de baja un préstamo, sus cuotas siguen en la base (no tienen baja lógica) pero dejan de poder consultarse.
- No existe control de titularidad ni de funciones: la misma credencial que solicita un préstamo puede aprobarlo y pagar sus cuotas.